PowerModels Mitigation Plan

Generated: 2026-03-08 Based on: vault-project-analysis-2026-03-08-v3.md Sources: Implementation vault (2026-03-08 rescan), accountant workflow proposal, process manager infrastructure, persisted RM infrastructure, corrected data-usage metrics, GitHub history (317 issues, 240 PRs, 9mo)

Thesis

PowerModels is transitioning from Shoebox (data import/reconciliation) to Accounting Phase 2 (journal, classification, period close). The v3 analysis identifies a **compound cycle**: every new aggregate, RM, and cross-aggregate workflow makes startup performance, defect rates, and saga coordination worse — unless the available infrastructure (checkpoint store, process manager base) is integrated first.

This plan sequences work to **break the cycle before expanding the domain**.

Phase 0: Safety Net (est. 1–2 weeks)

Goal: Establish the replay correctness foundation that every subsequent phase depends on.

0.1 RestoreFromEvents Test for ServerFinancialModel

Risk addressed	0/27 aggregates have RestoreFromEvents tests. SFM has 87 Apply methods, 3,557 lines, and is replayed on every business open. A silent Apply bug would corrupt state for every user.
Evidence	test-coverage.md: 0/27; god-aggregate.md: 87 Apply, 31 Register (2.8× ratio indicates overloaded handlers)
Deliverable	Test that creates SFM via command sequence → saves → replays from events → asserts equivalent state
Pattern established	First RestoreFromEvents test becomes the template for all 27 aggregates

0.2 Fix 4 Missing Source Constructors

Risk addressed	ServerFinancialModel, ReferenceDataSeries, DataSourceMapping, AccountingSystem cannot properly track command correlation. AccountingSystem is the next expansion target (accountant workflow).
Evidence	aggregates.md: 4 flagged
Deliverable	Add `:base(source)` constructor to each; verify existing tests pass

0.3 RestoreFromEvents Tests for Next-Growth Aggregates

Risk addressed	ChartOfAccounts (14 PRs, most active), AccountBalance (next growth area), and AccountingSystem (about to be extended) are all changing soon.

Evidence	feature-provenance.md: ChartOfAccounts 14 PRs; accountant workflow: extends AccountingSystem + AccountBalance
Deliverable	RestoreFromEvents tests for ChartOfAccounts, AccountBalance, AccountingSystem, EntrySet, TasklistItem

Phase 0 Exit Criteria

- [x] SFM RestoreFromEvents test passing
- [x] All 4 source constructors fixed
- [x] RestoreFromEvents tests for 6 aggregates (SFM + 5 growth targets)
- [x] CI green

Phase 0 Cumulative Impact

Metric	Before	After
RestoreFromEvents coverage	0/27	6/27 (God Aggregate + growth targets)
Missing source constructors	4	0
Replay corruption risk	Undetectable	Detected for 6 highest-risk aggregates

---

Phase 1: Infrastructure Integration (est. 3–4 weeks)

Goal: Wire the checkpoint and process manager infrastructure into PowerModels so that Phase 2 features build on solid ground.

1.1 Integrate ICheckpointStore into ReadModelBase

Risk addressed	46 ReadModelBase RMs replay from position 0 on every business open. A mature business with 15K events triggers ~135K handler invocations on the hottest stream alone.
Evidence	read-models.md: 46 ReadModelBase, 25 category streams; data-usage-and-storage.md: full replay on startup; ICheckpointStore exists in process-manager/read-model-caching-reference/
Deliverable	ReadModelBase accepts optional `ICheckpointStore`. On startup: `GetCheckpoint(name)` → resume from last position. On each event batch: `SaveCheckpoint` atomically.
Integration notes	ReadModelBase constructor already supports this pattern. Start with `InMemoryCheckpointStore` for testing, `PostgresCheckpointStore` for production. Requires adding PostgreSQL to the per-business or firm-level topology.

Migration pattern for existing RMs:

```
// BEFORE: No persistence, no checkpoints
public class ModelTemplateRm : ReadModelBase, IHandle<DataTableMapped>, ... {
    private readonly Dictionary<Guid, ModelTemplate> _templates = new();

    public ModelTemplateRm(IConfiguredConnection connection)
```

```
        : base("ModelTemplateRm", connection) {
        Start<ServerFinancialModel>(); // replays $ce-ServerFinancialModel from position 0
    }
}

// AFTER: Two-tier storage + checkpoint resume
public class ModelTemplateRm : ReadModelBase, IHandle<DataTableMapped>, ... {
    private readonly IReadModelStore<Guid, ModelTemplateState> _store;

    public ModelTemplateRm(
        IConfiguredConnection connection,
        IReadModelStore<Guid, ModelTemplateState> store,
        ICheckpointStore checkpoints)
        : base("ModelTemplateRm", connection, checkpoints) {
        _store = store;
        // ReadModelBase uses checkpoint to resume from last position (not position 0)
    }

    public void Handle(DataTableMapped @event) {
        _store.Upsert(@event.Id, new ModelTemplateState { ... });
        // Checkpoint saved by ReadModelBase infrastructure
    }
}
```

1.2 Checkpoint the 9 RMs on $ce-ServerFinancialModel

Risk addressed	Hottest stream: 9 subscribers × full SFM event history = dominant startup cost.
Evidence	read-models.md: ModelTemplateRm, FinancialModelRm, FinancialModelListRm, ModelListRm, ModelWorksheetTablesRm, DataTableFromTemplateRm, ModelVerificationRM, ModelsRm, SingleModelTaskMetricsRm
Deliverable	All 9 RMs wired with `ICheckpointStore`. Startup replays only events since last checkpoint.
Validation	Measure startup time before/after for a mature business (15K+ events). Target: >80% reduction in SFM stream handler invocations.

1.3 Integrate IReadModelStore for Two-Tier RM Persistence

Risk addressed	All RM state is in-memory only. Restart = full rebuild. Adding new RMs (Ledger, ReconciliationState, PeriodClose) without persistence compounds the problem.
Evidence	IReadModelStore + InMemoryReadModelStore + DapperReadModelStore exist in process-manager/read-model-caching-reference/
Deliverable	Hot path via `InMemoryReadModelStore` (ConcurrentDictionary). Periodic flush to `DapperReadModelStore` (PostgreSQL). On startup: load from DB, then replay only events since checkpoint.

Scope	Start with the 9 SFM RMs (highest value). Remaining 37 ReadModelBase RMs migrate incrementally.

1.4 Integrate ProcessManagerBase into PowerModels

Risk addressed	7 identified workflow chains, 0 explicit process managers. Reconciliation saga alone = 19 defects (29% of total). Accountant workflow adds 3+ new cross-aggregate workflows.
Evidence	saga-catalog.md: 7 implicit sagas; ProcessAggregateBase + ProcessManagerBase exist in process-manager/Infrastructure/
Deliverable	ProcessAggregateBase and ProcessManagerBase<TProcessAgg, TState> available as base classes in the PowerModels solution. NuGet or project reference wired. CategoryStreamWarmup + EventVersioning included.
Integration notes	Does NOT require rewriting existing sagas yet — just makes the infrastructure available. Phase 2 uses it for new sagas; Phase 3 retrofits existing ones.

1.5 Aggregate Snapshot Support for ServerFinancialModel

Risk addressed	Every `GetById(sfmId)` replays the full SFM event stream. With 100+ commands routing through FinancialModelService, this is the most frequently replayed aggregate.
Evidence	data-usage-and-storage.md: full stream replay per GetById, no caching; god-aggregate.md: 3,557 lines, 166 commands
Deliverable	`CachingRepository` wrapper for SFM. In-memory cache of `GetById` results. On cache miss: full replay. On cache hit: return cached instance (or replay from snapshot position).
Future	Persisted snapshot tier (relational DB) can be added later. In-memory caching alone eliminates redundant replays within a session.

Phase 1 Exit Criteria

- [x] ICheckpointStore integrated into ReadModelBase
- [x] 9 SFM RMs checkpointed — startup time measured before/after
- [x] IReadModelStore wired for SFM RMs (two-tier: in-memory + PostgreSQL)
- [x] ProcessManagerBase available in solution
- [x] CachingRepository wrapping SFM GetById
- [x] CI green, all Phase 0 tests still passing

Phase 1 Cumulative Impact

Metric	Before	After Phase 1
SFM stream startup invocations	9 × full history	9 × (events since last checkpoint)
RM state persistence	None	9 SFM RMs persisted
SFM GetById cost	Full stream replay every time	Cached within session

Explicit process managers	0 (infrastructure absent)	0 (infrastructure available)
RestoreFromEvents coverage	6/27	6/27

Phase 2: Domain Foundation (est. 4–6 weeks)

Goal: Build the accounting domain foundation using the infrastructure from Phase 1. New aggregates, new sagas, new RMs — all with checkpoints and explicit coordination from day one.

2.1 Extend AccountingSystem (Epic 1.1–1.3)

Gap addressed	AccountingSystem is a 29-line shell with only `AccountingSystemCreated`. The accountant workflow requires entity type, industry, fiscal year, and close policy.
Evidence	small-accountant-flow-proposal.md: Epic 1 (Company Setup); aggregates.md: 29 lines
Deliverable	New commands + events: `SetEntityType`, `SetIndustry`, `SetFiscalYear`, `SetClosePolicy`. Source constructor already fixed in Phase 0.
Tests	RestoreFromEvents (already exists from Phase 0), new idempotency + state guard tests for each command

2.2 Create Journal Aggregate (Epic 3.1)

Gap addressed	No Journal aggregate exists. Every other accounting feature (GL reports, reconciliation, period close) depends on journal entries.
Evidence	small-accountant-flow-proposal.md: Epic 3.1; domain-model.md: EntrySet is manual entries only, no balanced debits/credits
Deliverable	Journal aggregate with: `PostEntry` (balanced debit/credit), `ReverseEntry` (compensating), period-aware validation (against fiscal year from AccountingSystem)
Tests	RestoreFromEvents, idempotency, state guard — established pattern from Phase 0
Infrastructure	New `$ce-Journal` category stream. Ledger RM subscribing with ICheckpointStore from day one.

Design constraints — bus-only coordination:

- Other contexts interact with Journal via **commands on the bus**, not direct `GetById`
- Classification pipeline sends `PostEntry` command; reconciliation sends `PostEntry` for adjustments
- No handler should call `GetById` on Journal AND another aggregate in the same handler method — this prevents creating implicit saga #8
- **Ledger RM placement:** Build in **ModelServer**, not SpreadsheetAdapter. SpreadsheetAdapter projection only when UI requires it. This keeps the defect funnel from growing.

Data usage impact: Moderate — journal entries are lower volume than data imports (~50-200 entries/month vs ~1,001 events per CSV import)

2.3 Industry CoA Template Engine (Epic 2.1–2.2)

Gap addressed	No industry-driven chart of accounts templates.
Evidence	small-accountant-flow-proposal.md: Epic 2; ChartOfAccounts is already 540 lines with account hierarchy
Deliverable	`SeedFromTemplate(entityType, industry, basis)` command on ChartOfAccounts. Template data as configuration, not hard-coded.

2.4 Explicit Reconciliation ProcessManager

Risk addressed	#1 defect generator: 19 defects (29%), 5 contexts, 5 aggregates, implicit coordination, no compensation.
Evidence	defect-analysis.md: 19/66 defects; saga-catalog.md: Reconciliation saga details; ProcessManagerBase now available from Phase 1
Validation	Compare defect rate for reconciliation workflows before/after. Target: <10% defect rate vs current 29%.

Process Aggregate: `ReconciliationProcess : ProcessAggregateBase<ReconState>`

State machine:

```
enum ReconState {
    NotStarted,         // default — process not yet initiated
    Importing,          // statement data being parsed/validated
    Matching,           // transactions being matched to journal entries
    ReviewPending,      // unmatched items awaiting accountant review
    AdjustmentsPending,// reconciliation adjustments queued for posting
    Completing,         // final balance comparison in progress
    Reconciled,         // terminal — statement balance = book balance
    Failed              // terminal — unrecoverable error or user abort
}
```

State handler sketch:

```
protected override IReadOnlyList<ReconState> TerminalStates => [ReconState.Reconciled,
ReconState.Failed];

protected override void RegisterStateHandlers() {
    InState(ReconState.NotStarted)
        .On<ReconciliationStarted>(e => {
            Raise(new ReconStateChanged(Id, ReconState.Importing));
            IssueCommand(new ParseStatementData(e.AccountId, e.StatementId));
        });

    InState(ReconState.Importing)
        .On<StatementDataParsed>(e => {
            Raise(new ReconStateChanged(Id, ReconState.Matching));
            IssueCommand(new MatchTransactions(Id, e.AccountId));
```

```
        })
        .On<StepTimedOut>(_ => {
            Raise(new ReconStateChanged(Id, ReconState.Failed));
            Raise(new ReconciliationFailed(Id, "Statement import timed out"));
        });

    InState(ReconState.Matching)
        .On<TransactionsMatched>(e => {
            if (e.UnmatchedCount > 0) {
                Raise(new ReconStateChanged(Id, ReconState.ReviewPending));
            } else {
                Raise(new ReconStateChanged(Id, ReconState.Completing));
                IssueCommand(new CompareBalances(Id, e.AccountId, e.PeriodEnd));
            }
        });

    InState(ReconState.ReviewPending)
        .On<ReviewCompleted>(e => {
            if (e.HasAdjustments) {
                Raise(new ReconStateChanged(Id, ReconState.AdjustmentsPending));
                foreach (var adj in e.Adjustments)
                    IssueCommand(new PostEntry(adj.JournalId, adj.Pattern, adj.Lines));
            } else {
                Raise(new ReconStateChanged(Id, ReconState.Completing));
                IssueCommand(new CompareBalances(Id, e.AccountId, e.PeriodEnd));
            }
        });

    InState(ReconState.AdjustmentsPending)
        .On<AdjustmentsPosted>(_ => {
            Raise(new ReconStateChanged(Id, ReconState.Completing));
            IssueCommand(new CompareBalances(Id, ...));
        });

    InState(ReconState.Completing)
        .On<BalancesCompared>(e => {
            if (e.IsBalanced) {
                Raise(new ReconciliationCompleted(Id, e.AccountId, e.Period,
e.EndingBalance));
                Raise(new ReconStateChanged(Id, ReconState.Reconciled));
            } else {
                Raise(new ReconStateChanged(Id, ReconState.ReviewPending));
                Raise(new BalanceMismatchDetected(Id, e.Difference));
            }
        });
}
```

Before/after capability comparison:

Capability	Before (implicit)	With ProcessAggregateBase

State visibility	None — poll SpreadsheetAdapter RMs	`State` property on aggregate, persisted via events
Timeout handling	None — imports hang indefinitely	`CreateStepTimeout()` with auto-staleness detection
Compensation	None — ~1,001 events committed with no rollback	`IssueCommand` for compensating entries on failure
Retry	None — restart from scratch	`Failed` state preserves context; re-send `StartReconciliation`
Audit trail	Scattered across handler logs	`CommandsIssued` event serializes all dispatched commands
Replay safety	N/A	Event-sourced aggregate replays via `Register<T>(Apply)`

Process Manager: `ReconciliationProcessManager : ProcessManagerBase<ReconciliationProcess, ReconState>`

```
public ReconciliationProcessManager(
    IConfiguredConnection connection,
    IRepository repository,
    IBus bus)
    : base("ReconciliationPM", connection, repository, bus) {

    EventStream.Subscribe<StatementDataParsed>(this);
    EventStream.Subscribe<TransactionsMatched>(this);
    EventStream.Subscribe<ReviewCompleted>(this);
    EventStream.Subscribe<AdjustmentsPosted>(this);
    EventStream.Subscribe<BalancesCompared>(this);

    Start<ReconciliationProcess>(blockUntilLive: true);
    GoLive();
}

public void Handle(StatementDataParsed @event) {
    var (agg, _) = LoadOrCreate(@event.ProcessId);
    agg.Source = NewCorrelationSource();
    agg.When(@event);
    var commands = SaveAndTakeCommands(agg);
    DispatchCommands(commands);
}
```

New streams: 1 per reconciliation instance (`$ce-reconProcessAgg` , requires `CategoryStreamWarmup`) **New RMs:** 0 — the ProcessManager IS the read model (extends ReadModelBase)

Migration: Existing Pipeline → ProcessManager (Phase 2.4b)

The existing reconciliation pipeline (Pipeline.cs, 12+ steps) must be wired through the new ProcessManager:

1. Add ProcessManager infrastructure to PowerModels (project reference or copy from `process-manager/Infrastructure/`)
2. Add `StepTimedOut` and `CommandsIssued` events to domain messages
3. Create `ReconciliationProcess` aggregate + `ReconciliationProcessManager`
4. Add `CategoryStreamWarmup.Warmup(connection, ["reconProcessAgg"])` to composition root
5. Pipeline.start → sends command that creates/starts `ReconciliationProcess`
6. Each pipeline step outcome → domain event → ProcessManager routes to aggregate → state transition + next command
7. Pipeline.complete → `ReconciliationCompleted` event from aggregate
8. SpreadsheetAdapter RMs that track reconciliation state → subscribe to `ReconciliationProcess` events instead
9. **Feature flag:** Route through ProcessManager only when flag is on; keep old pipeline as fallback until verified end-to-end

This is where the 19 reconciliation defects get addressed structurally — not by fixing each bug individually, but by replacing the implicit coordination with a formal state machine.

2.5 SpreadsheetAdapter RM Audit

Risk addressed	40 RMs in SpreadsheetAdapter, 15% defect rate, 13/16 defect-hit RMs are here. New accounting features add more RMs to this context.
Evidence	bounded-contexts.md: 40 RMs; defect-analysis.md: 15% rate
Deliverable	Audit all 40 RMs: identify overlapping subscriptions, consolidation candidates, unused RMs. Migrate remaining high-value RMs to checkpoint + persistence.

Phase 2 Exit Criteria

- [x] AccountingSystem extended with entity type, industry, fiscal year, close policy
- [x] Journal aggregate created with balanced entries and period validation
- [x] CoA template seeding working
- [x] Reconciliation ProcessManager operational with compensation and timeouts
- [x] SpreadsheetAdapter RM audit complete, top consolidation candidates identified
- [x] All new aggregates have RestoreFromEvents + idempotency + state guard tests
- [x] All new RMs wired with ICheckpointStore

Phase 2 Cumulative Impact

Metric	Before	After Phase 2
Missing aggregates (accountant workflow)	3	1 (ReconciliationState created via ProcessManager; PeriodClose remains)
Explicit process managers	0	1 (Reconciliation)
Reconciliation defect rate	29%	Target <10%
AccountingSystem completeness	1 event (Created)	5+ events (entity type, industry, fiscal year, close policy)

Journal capability	None (EntrySet only)	Full journal with balanced entries
RestoreFromEvents coverage	6/27	9/27+ (+ Journal, AccountingSystem re-validated, ReconciliationProcessAggregate)

Phase 3: Feature Buildout (est. 4–6 weeks)

Goal: Complete the accountant workflow epics and address remaining architectural debt.

3.1 ReconciliationState + PeriodClose Aggregates (Epic 5.1–5.2)

Gap addressed	Bank reconciliation state tracking and period close workflow.
Evidence	small-accountant-flow-proposal.md: Epic 5; ReconciliationProcessManager (Phase 2) coordinates but needs dedicated state aggregates
Deliverable	ReconciliationState (per account per period: unreconciled items, matched pairs, adjustments). PeriodClose (soft lock, hard lock, unlock with audit trail).
Infrastructure	New `$ce-ReconciliationState` and `$ce-PeriodClose` streams. RMs with ICheckpointStore.

Period Close Process Manager: `PeriodCloseProcess : ProcessAggregateBase<CloseState>`

```
enum CloseState {
    Open,              // default — period accepts entries
    ChecklistPending,  // close initiated, checklist items being verified
    SoftClosed,        // warnings on new entries but not blocked
    HardClosed,        // new entries blocked
    Reopened           // terminal (reverts to Open on next cycle)
}
```

Coordination via commands:

- `SoftClose` → `IssueCommand(new GenerateCloseChecklist(...))` → TasklistItem aggregate creates checklist tasks
- `HardClose` → validates all reconciliations complete (queries ReconciliationProcess RM), all checklist items done
- `ReopenPeriod` → `IssueCommand(new PostEntry(reopeningAdjustment))` if needed
- Period close status queryable by Journal aggregate's validation layer to reject entries to closed periods

Recon Adjustments (Epic 5.3):

- Reconciliation adjustment patterns (bank fees, interest, NSF) are Entry Patterns (3.2) that produce `PostEntry` commands
- ReconciliationProcess issues these during `AdjustmentsPending` state
- Flow: ReconciliationProcess → `IssueCommand(PostEntry)` → Journal aggregate → Ledger RM updates
- No handler calls `GetById` on both Journal and ReconciliationProcess — command dispatch is the only coordination

New streams: 1 per business per period (`$ce-periodCloseAgg` , requires `CategoryStreamWarmup`)

3.2 Entry Patterns Library (Epic 3.2)

Gap addressed	No pre-defined journal entry templates. Manual entries require full understanding of debit/credit mechanics.
Evidence	small-accountant-flow-proposal.md: Epic 3.2
Deliverable	Library of entry patterns: Purchase, Revenue, Payroll, Depreciation, Loan Payment, etc. Each pattern maps to Journal commands with pre-filled debit/credit accounts.

3.3 Classification Rules Enhancement (Epic 4.1–4.2)

Gap addressed	Rules are hardcoded keyword-matching only. No per-entity configurability or 85/15 threshold enforcement.
Evidence	small-accountant-flow-proposal.md: Epic 4; RuleStep.cs exists with hardcoded patterns
Deliverable	Per-entity configurable rules. 85/15 confidence threshold: ≥85% auto-approve, <85% route to review queue.

3.4 GL Report Generation (Epic 3.3)

Gap addressed	Trial balance, income statement, balance sheet only available via QuickBooks import.
Evidence	small-accountant-flow-proposal.md: Epic 3.3
Deliverable	Read model projections from Journal events → Trial Balance RM, Income Statement RM, Balance Sheet RM. All with ICheckpointStore.

3.5 Extract Table-Mapping from ServerFinancialModel

Risk addressed	20 table-mapping events registered in SFM that already have dedicated aggregates (DataTableMap, ListDataTableMap, ManualTableMap). These events inflate the SFM stream and couple mapping logic to the God Aggregate.
Evidence	god-aggregate.md: 20 extractable events; saga-catalog.md: Financial Model Mapping workflow
Deliverable	Move DataTableMapped, DataTableMapRowAdded, ListDataTableMapped, ManualTableMapped (etc.) event registration out of SFM. Routing through existing dedicated aggregates.
Impact	Reduces SFM registered events from 31 to ~11. Reduces SFM Apply methods from 87 to ~67. Shrinks the hottest category stream.

3.6 RestoreFromEvents Tests for All Remaining Aggregates

Risk addressed	18/27 still untested after Phase 2.
Deliverable	Batch-write RestoreFromEvents tests for all remaining aggregates using the established pattern from Phase 0.

Phase 3 Exit Criteria

- [x] ReconciliationState + PeriodClose aggregates created and tested
- [x] Entry patterns library operational
- [x] Classification rules configurable per entity with 85/15 threshold
- [x] GL reports (trial balance, income statement, balance sheet) rendering from Journal events
- [x] 20 table-mapping events extracted from SFM
- [x] RestoreFromEvents tests for 27/27 aggregates
- [x] All new RMs wired with ICheckpointStore

Phase 3 Cumulative Impact

Metric	Before Plan	After Phase 3
RestoreFromEvents coverage	0/27	27/27 + 3 new aggregates
Missing source constructors	4	0
Explicit process managers	0	1+ (Reconciliation; Business Setup candidate)
RM checkpoint coverage	0/46	9+ SFM RMs + all new RMs
SFM registered events	31	~11 (20 extracted)
SFM Apply methods	87	~67 (20 extracted)
Accountant workflow epics complete	0/5	5/5
Missing aggregates	3 (Journal, ReconciliationState, PeriodClose)	0
Reconciliation defect rate	29%	Target <10%

Phase 4: Consolidation (est. 2–3 weeks)

Goal: Solidify gains, extend checkpoint coverage, retrofit remaining implicit sagas.

4.1 Checkpoint All Remaining ReadModelBase RMs

Extend ICheckpointStore to all 46 ReadModelBase RMs (37 remaining after Phase 1's 9).

4.2 Explicit Business Setup ProcessManager

Second-highest-risk implicit saga (3 defects, multi-step creation workflow). Use Phase 2's reconciliation ProcessManager as template.

4.3 Audit ModelTemplateRm (82 Event Subscriptions)

Determine if ModelTemplateRm truly needs to subscribe to 82 events (19% of all event types). Identify events that can be dropped or split into a secondary RM.

4.4 Split FinancialModelService

With 20 table-mapping events extracted from SFM (Phase 3.5), split FinancialModelService into:

- FinancialModelService (core model operations, ~100 commands)
- TableMappingService (data/manual/list table mapping, ~60 commands)

4.5 Measure and Baseline

Measurement	Method	Target
Startup time	Instrument business open for 1K, 5K, 15K, 50K event businesses	<2s for 15K events
Command latency	Instrument GetById for SFM at various stream sizes	<100ms for 5K events
Defect rate	Compare Phase 3 milestone defects vs historical	<10% overall
RM replay invocations	Count handler invocations on startup with checkpoints	>90% reduction vs pre-Phase 1

Cumulative Summary Tables

Saga Explicitness (Before → After)

Saga	Before	After Plan
Reconciliation	Implicit (19 defects, 5 contexts)	**ProcessManager** — `ReconciliationProcess : ProcessAggregateBase<ReconState>` with 8 states, timeouts, compensation
Period Close	Doesn't exist	**ProcessManager** — `PeriodCloseProcess : ProcessAggregateBase<CloseState>` coordinating close workflow
Journal → Recon → Close	Doesn't exist	**Explicit from day one** — ProcessManager → IssueCommand → DispatchCommands
Data Import Pipeline	Implicit (8 defects)	Implicit (Phase 4+ candidate)
Business Setup	Implicit (3 defects)	Phase 4.2 candidate — use reconciliation PM as template
Financial Model Mapping	Implicit (coupling)	Reduced — 20 events extracted from God Aggregate in Phase 3.5
Classification Pipeline	Implicit	Partially explicit — Journal integration via bus commands

New Aggregate Summary

Aggregate	Base Class	Phase	Context	Streams
Journal	AggregateRoot	2.2	ModelServer/AS	1/business
ReconciliationProcess	ProcessAggregateBase<ReconState>	2.4	ModelServer/AS	1/recon instanc
PeriodCloseProcess	ProcessAggregateBase<CloseState>	3.1	ModelServer/AS	1/business/peri
ReconciliationState	AggregateRoot	3.1	ModelServer/AS	1/account/perio
PeriodClose	AggregateRoot	3.1	ModelServer/AS	1/business/peri

Key difference: ProcessAggregates are coordinated by ProcessManagers (which extend ReadModelBase), not by handler services. The ProcessManager IS the read model — no separate RM needed. All new read-side projections placed in ModelServer; SpreadsheetAdapter projections only when UI demands.

Dependencies and Risks

External Dependencies

Dependency	Phases Affected	Risk	Mitigation
PostgreSQL addition to topology	1.1, 1.2, 1.3	Medium — new database in production	Start with InMemoryCheckpointStore; PostgreSQL optional for initial integration
ReactiveDomain CachingRepository	1.5	Low — exists in R-D but may need extension	In-memory cache is sufficient for Phase 1
ProcessManagerBase namespace (Greylock.Domain.Infrastructure)	1.4, 2.4	Low — reference code, may need namespace adjustment	Adapt namespace to PowerModels.Domain.Infrastructure
March demo deadline (#2099, #2106-2109)	0, 1	Medium — Phase 0 competes with active demo work	Phase 0 source ctor + RestoreFromEvents tests don't touch demo-critical code paths
joshkempner availability (69% of PRs)	All	Medium — primary implementer for all phases	Phase 2 tracks parallelizable across 2+ developers
Existing reconciliation pipeline stability	2.4b	Medium — Phase 2.4b	Keep old pipeline as fallback until new path is verified end-to-end (feature

		rewires a working pipeline	flag)

Technical Risks

Risk	Likelihood	Impact	Mitigation
RestoreFromEvents tests reveal Apply bugs in SFM	HIGH	HIGH — could require event versioning	EventVersioning + VersionedEventSerializer available in process-manager/Infrastructure/. Fix Apply bugs, add upcasters for schema changes.
Checkpoint integration changes ReadModelBase constructor signature	MEDIUM	MEDIUM — affects 46 RMs	Optional parameter with null default. Opt-in migration.
Journal aggregate design doesn't accommodate all entry types	MEDIUM	HIGH — foundational dependency	Start with minimal Entry (debit/credit/amount/date), extend incrementally. Entry Patterns Library (Phase 3.2) validates coverage.
SFM table-mapping extraction breaks existing read models	MEDIUM	HIGH — 9 subscribing RMs	Run dual-write period: events on both old (SFM) and new (dedicated) streams. Migrate RMs one at a time. Use EventVersioning for schema evolution.
AccountingSystem grows into configuration God Aggregate	LOW	MEDIUM — overloads configuration responsibility	Review at Phase 2 exit — if >200 lines, extract close policy to PeriodClose
Snapshot deserialization breaks on aggregate schema change	MEDIUM	MEDIUM — cache invalidation	Invalidate snapshots on version mismatch — fall back to full replay. Snapshot version tracked alongside stream version.
PostgreSQL RM state diverges from ESDB truth	LOW	HIGH — stale projections	`ICheckpointStore` ensures exactly-once position tracking; `ExecuteTransactional()` for atomic state+checkpoint saves; rebuild from stream always available as fallback

---

Success Metrics

Metric	Current	Phase 1 Target	Phase 3 Target
Startup time (15K events)	Unknown (unmeasured)	Measurable baseline	<2s
RestoreFromEvents	0/27	6/27	27/27+
Explicit process managers	0	0 (infra available)	2 (Reconciliation, PeriodClose)

RM checkpoint coverage	0%	9/46 (20%)	9+ SFM + all new
SFM lines	3,557	3,557	~1,500-2,000 (after extraction)
SFM registered events	31	31	~11 (20 extracted)
SFM Apply methods	87	87	~67
SFM GetById cost	Full stream replay	Cached within session	<100 events (snapshot)
FinancialModelService commands	166	166	~80-90 (mapping split to TableMappingService)
Reconciliation defect rate	29%	29%	<10%
SpreadsheetAdapter defect rate	15%	15%	<10%
Implicit sagas	7	7	≤4 (Reconciliation + PeriodClose + Journal→Recon→Close explicit)
Accountant workflow completeness	0/5 epics	0/5	5/5
Missing aggregates	3	3	0
New aggregate test compliance	N/A	N/A	100% (RestoreFromEvents + source ctor + idempotency + state guards)
ProcessManager pattern compliance	N/A	N/A	100% of new sagas use ProcessAggregateBase + ProcessManagerBase
Cross-aggregate coordination	Mixed (GetById + bus)	Mixed	100% of new interactions via bus commands only

---

Vault Integration

Each phase produces artifacts that feed back into the vault:

Phase	Updated Docs	New Docs
0	test-coverage.md (RestoreFromEvents counts), aggregates.md (source ctors fixed)	—
1	read-models.md (checkpoint status), data-usage-and-storage.md (startup metrics)	infrastructure-integration.md (checkpoint + PM wiring)
2	saga-catalog.md (reconciliation → explicit), bounded-contexts.md (new aggregates), feature-provenance.md (new PRs)	journal-aggregate.md, reconciliation-process-manager.md

3	god-aggregate.md (reduced events), message-map.md (split handler)	gl-reports.md, entry-patterns.md
4	All metrics docs updated with post-plan baselines	performance-baseline.md

Rescan after each phase to keep the vault current. The VaultTool scanner will automatically detect new aggregates, RMs, and handler changes.

Drift Detection

Run after each phase:

```
dotnet run -- project-refresh --config ../../projects/PowerModels/graph-vault.json
```

Expected drift signals:

- **Phase 0:** No drift (tests + ctor fixes don't change architecture docs)
- **Phase 1:** Drift in read-models.md (checkpoint + store changes), application-topology.md (PostgreSQL dependency added), data-usage-and-storage.md (startup metrics)
- **Phase 2:** Drift in aggregates.md (Journal, ReconciliationProcess), bounded-contexts.md, saga-catalog.md (reconciliation → explicit), message-map.md (new commands)
- **Phase 3:** Drift in god-aggregate.md (reduced events), message-map.md (split handler), aggregates.md (ReconciliationState, PeriodClose)
- **Phase 4:** All metrics docs updated with post-plan baselines